

K DEEP SIMPLEX - A LOCAL DICTIONARY AND SPARSE LEARNING ALGORITHM: APPLICATION FOR PRICE PREDICTION OF USED CARS

TARIK KRESTALICA, JULIA DOLCE, AND PABLO HERRERA

1. INTRODUCTION

Supervised tasks like price prediction with high-dimensional and heterogeneous data present both statistical and interpretability challenges when choosing an effective model. Principal Component Analysis(PCA) and Non-negative Matrix Factorization(NMF) are better-known techniques used for data compression, but both have trade-offs when it comes to predictive performance and interpretability. Recently, K-Deep Simplex(KDS) has been proposed as a flexible and sparse alternative that aims to preserve feature structure, making the findings very interpretable, while also supporting supervised learning tasks. In this study, we performed PCA, NMF, and KDS on a large data set to compare their reconstruction quality, predictive accuracy, and interpretability with a focus on whether KDS can outperform these better-known dimension reduction techniques in interpretability and accuracy.

2. PCA AND ITS LIMITATIONS

While Principal Component Analysis provides a rigid and deterministic decomposition of the dataset, it remains one of the most widely used linear dimensionality-reduction methods. First, PCA starts by centering a data matrix X , and then performing SVD on this centered data matrix:

$$(2.1) \quad X' = U\Sigma V^\top.$$

The right singular vectors in V are the orthonormal principal components, and the singular values along the diagonal of Σ determine how much variance each component explains. The variance that the first f principal components explain of d total principal components can be denoted by:

$$(2.2) \quad \frac{\sum_{i=1}^f \sigma_i^2}{\sum_{i=1}^d \sigma_i^2},$$

PCA always yields the same order components and produces dense loadings that don't emphasize locality or sparsity. It is known to perform well for predictive accuracy when a moderate amount of the first principal components captures significant variance in the data.

However, PCA has several limitations when it comes to feature interpretation or supervised tasks like price prediction. PCA is sensitive to outliers, dimensionality reduction always involves some information loss, and it is strictly linear, so it can miss nonlinear relationships in real-world data. Most notably, PCA transforms original variables into uncorrelated linear combinations, making interpretability and feature importance analysis difficult. This loss

of meaning when switching bases to the principal components can make downstream modeling difficult and obscure what original features drive price predictions. These limitations motivate a more flexible and interpretable model to be introduced for supervised tasks.

3. NMF AND ITS LIMITATIONS

Given a nonnegative data matrix $A \in \mathbb{R}^{n \times d}$, NMF seeks a low-rank approximation of the form $A \approx WH$, where $W \in \mathbb{R}^{n \times r}$ contains the low-dimensional embeddings of each observation, $H \in \mathbb{R}^{r \times d}$ contains the learned latent components, and r is a user-selected rank controlling model complexity. The optimization formula is:

$$(3.1) \quad \frac{1}{2} \|A - WH\|_F^2 = \sum_{i=1}^n \sum_{j=1}^m (A_{ij} - (WH)_{ij})^2$$

This is usually solved with multiplicative updates or alternating least squares [4]. Unlike PCA, NMF restricts both W and H to be nonnegative. It is this constraint that allows for the parts-based representation that are easier to interpret, especially when input features are also nonnegative variables, such as one-hot encodings of manufacturers, regions or fuel types.

It is important to still consider an important NMF limitation in this context, which is that it does not include the regression target in its objective function. Therefore, nothing in the matrix factorization forces components to be predictive of price, even if they are capturing meaningful structures in the data.

4. INTRODUCTION TO KDS

Let $Y = [y_1, y_2, \dots, y_n] \in \mathbb{R}^{d \times n}$ be a set of d -dimensional data-points. For each $y_i \in \mathbb{R}^d$, we express it as a convex combination of the m archetypes with $m \ll n$. We define a dictionary $A = [a_1, a_2, \dots, a_m] \in \mathbb{R}^{d \times m}$, where each $a_i \in \mathbb{R}^d$ is an learned archetypes. This leads to $Y \approx AX$, where $X = [x_1, x_2, \dots] \in \mathbb{R}^{m \times n}$ is a coefficient matrix assigning weights to each of the archetypes. Each $x_i \in \mathbb{R}^m$ satisfies $\sum_{j=1}^m (x_i)_j = 1$ with $(x_i)_j \geq 0$. This provides us with an interpretable framework where the probabilistic coefficients represent how much weight each archetype contributes to the construction of y_i .

To ensure that we achieve a proper reconstruction of our data matrix Y , with each y_i using its nearby archetypes, we have the following regularization term $\sum_{i,j} (x_i)_j \|y_i - a_j\|^2$, with $(x_i)_j$ representing the j th entry of x_i .

This leads us to the following optimization problem.

$$(4.1) \quad \min_{A \in \mathbb{R}^{d \times m}, X \in \mathbb{R}^{m \times n}} \sum_{i,j} (x_i)_j \|y_i - a_j\|^2$$

subject to the condition $Y = AX$ with $\sum_{j=1}^m (x_i)_j = 1$ with $(x_i)_j \geq 0$ indicating how much weight archetype a_j contributes to constructing y_i .

4.1. Dictionary Learning Algorithm. This balancing of utilizing the m archetypes to find the local archetypes that effectively reconstruct each data point can be described of estimating the sparse matrix and dictionary atoms. That is, given a set of n data points, we study the following minimization problem.

$$(4.2) \quad \min_{A \in \mathbb{R}^{d \times m}, X \in \mathbb{R}^{m \times n}} \sum_{i=1}^n \left[\frac{1}{2} \|y_i - Ax_i\|^2 + \lambda \sum_{j=1}^m (x_i)_j \|y_i - a_j\|^2 \right]$$

where below is the reconstruction loss term

$$(4.3) \quad \sum_{i=1}^n \frac{1}{2} \|y_i - Ax_i\|^2$$

where below is the locality loss term.

$$(4.4) \quad \lambda \sum_{j=1}^m (x_i)_j \|y_i - a_j\|^2$$

The parameter λ controls the reconstruction loss and the locality regularization. This will be a critical discussion point as we training pipeline.

5. INTRODUCTION TO THE DATASET

The dataset used to explore our research question is the “Used Car Dataset” from Kaggle, which contains about 425,000 car listings scraped from Craigslist. It includes numeric and categorical features relevant to car value, like manufacturer, region, odometer, transmission, drive, condition, and fuel. For those categorical variables, we selected the 10 most frequent values and the values outside the 10 most frequent values are placed in an ‘other’ category. Before using this data set for modeling, this pre-processing pipeline was used to ensure that data quality was consistent over all variables and suitable for dimensionality reduction techniques. Numeric values were filled with the median of the column, and categorical values were replaced with the most common category of the column. Irrelevant columns like URLs and descriptions were removed, and the categorical variables followed the encoding process. In addition, two more features were derived to observe important relationships in the data: “price_per_mileage” and “mileage_per_year”.

The processed dataset was then split into training (10,000 points), validation (3,000 points), and test (5,000 points) sets, with the features denoted as “X” and car price denoted as “Y”. All the features and the price predictor variable were scaled using the Quantile Transformer to standardize distributions across all variables and ensure scale would not interfere with analysis. The pre-processing pipeline prepared the dataset for dimensionality reduction and allowed us to explore our predictive modeling techniques of PCA, KDS, and NMF.

6. GOALS OF THE PROJECT

The primary goal of this study is to evaluate the effectiveness of vehicle prediction using KDS, with PCA and NMF as better-known comparison techniques. These were chosen specifically because PCA provides a strong baseline for predictive performance, and NMF offers a more interpretable parts-based representation. After applying KDS, PCA, and NMF, each model’s learned features were used to perform price prediction, allowing an evaluation of data reconstruction quality and predictive performance. Since KDS produces sparse feature representations, we were interested in exploring how well this technique could identify

meaningful patterns in the data and reduce noise from less informative variables. To quantitatively compare each of these methods, we used RMSE and R^2 as performance metrics, using the formulas listed below:

$$(6.1) \quad R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

$$(6.2) \quad \text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

This analysis enables a clear comparison of how KDS, PCA, and NMF capture underlying structure in the data and contribute to robust predictive performance, helping to evaluate if KDS achieves the interpretability-prediction balance that motivates its use.

7. RESULTS

7.1. Principal Components Analysis. To evaluate PCA for vehicle price prediction, SVD was applied to the preprocessed data set, and a full set of PCs and singular values was extracted. After examining the variance explained by the curve, 20 principal components were selected as this subset retained over 80% of the total variance while still reducing the dimensionality of the original feature space. Principal component 1 only accounts for 16% of the variance in the data with a smooth drop-off, indicating that this dataset does not have a single dominating source of variation. Using these 20 components to reconstruct the data and train the regression model yielded an RMSE of 0.239 and R^2 of 0.3410, indicating that PCA captures a meaningful portion of the thesis datasets' structure and provides a reasonable predictive accuracy. To further explore the patterns captured by PCA, multiple figures were constructed to examine component sparsity and influential features, with two figures chosen for this paper to illustrate these characteristics.

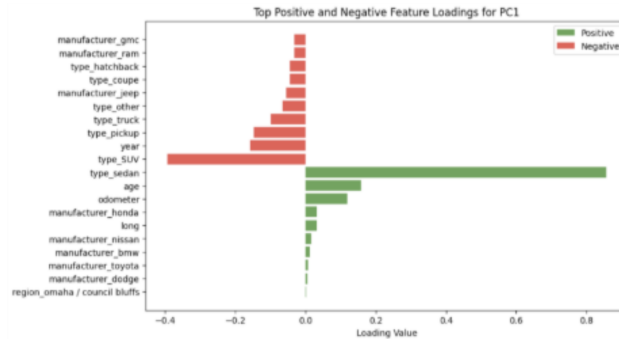


Figure 7.1.1: Top Positive and Negative Feature Loadings for PC1

This figure demonstrates which attributes drive the primary axis of variation in the dataset. This principal component seems to focus on differentiating lower-priced sedans or high-mileage vehicles from newer, higher-priced SUVs and trucks. Sedans influence the first principal components in the negative direction, while SUVs and trucks influence the first principal component in the positive direction, capturing a major structural pattern relevant to how this model performs price prediction. Sedans in this dataset tend to be slightly older, with a mean age of 14.6 years and a median age of 12 years, while trucks and SUVs have mean ages of 14 and 12.4 years, with a median age of 11 years. Sedans dominate the data

set with 180,000 vehicles, while there are 35,000 trucks and 77,000 SUVs. These age and count differences likely contribute to why the type of car pulls the first principal component in different directions, showing the primary structural variation in the dataset that drives price differences.

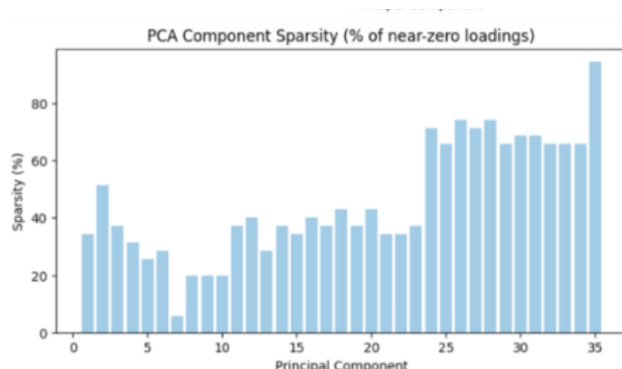


Figure 7.1.2: Sparsity Pattern of all Principal Components (percentage of loadings near zero)

Although PCA is a notable dense method of dimensionality reduction, this sparsity distribution provides insight into how each component uses the underlying features and how many features a principal component includes. Principal components 1-10 show low sparsity, indicating that they rely on multiple features to capture complex multi-variable relationships. Principal components 11-23 become less sparse, indicating that they are beginning to narrow down and focus more on specific groups of features. At principal component 24, there is a major jump in sparsity, as these components rely on less than 30% of the features. This visualizes how PCA with the vehicle dataset shifts from looking at the broad global structure in the early components to early specialized patterns after the 24th principal component.

These analyses show that PCA is an effective dimensionality reduction technique for this dataset. The first figure shows which features drive the primary axis of variation, with vehicle type, age, and mileage having the most effect on price. The second figure shows how the earlier principal components capture complex multivariable relationships of features, while later ones focus on more specific variables. Although PCA components are not as interpretable as the sparser archetype representations in KDS, they were still moderately effective in predicting prices with this dataset.

7.2. Non-negative Matrix Factorization. For this portion of the report, we will be examining Nonnegative Matrix Factorization (NMF) as a dimensionality reduction technique for the used car dataset, with emphasis on the interpretability rather than predictive functionality. Unlike variance maximizing techniques like PCA, NMF decomposes a nonnegative data matrix into additive parts, making it great for discovering interpretable latent structure in tabular datasets with many categorical indicators. The objective of this portion is twofold (i) determine whether NMF yields components that meaningfully summarize the structure of the used-car feature space, and (ii) determine whether those components can be used as effective predictors of vehicle price when used as input in a regression model.

The results obtained from this exploration are clear: NMF successfully produces interpretable components that align with intuitive feature attribute clusters in the dataset, yet these same components perform rather poorly on the price prediction task. This result shows

that interpretability and predictive accuracy are not always aligned, specifically for models such as NMF where we are optimizing reconstruction error, rather than a target.

The actual analysis in the dataset starts with necessary data pre-processing. For starters all textual or irrelevant columns were removed, categorical variables were encoded with one-hot vectors, and numerical variables such as odometer, mileage-per-year were scaled. The price column was dropped entirely prior to constructing the NMF input matrix to avoid any leakage. This ensures that the factorization is based only on vehicle attributes rather than dependent variables.

We fit NMF across a range of different ranks. To select the appropriate/optimal value, we computed the reconstruction error with

$$(7.1) \quad \|X - WH\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n (X_{ij} - (WH)_{ij})^2}.$$

for various values of r . The table below shows a plot of reconstruction error across ranks. The curve shows the expected diminishing-returns pattern, and we selected $r = 50$ using the standard elbow-method heuristic. Everything following will use the corresponding W matrix as a feature representation.

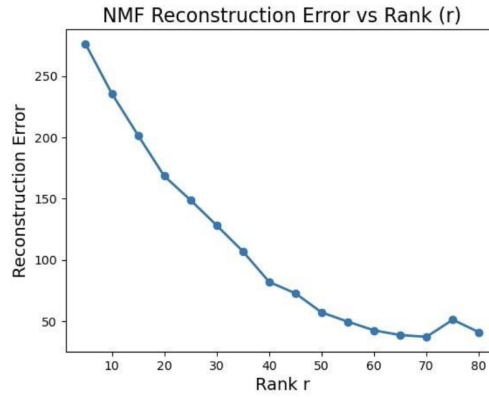


Figure 7.2.1: Reconstruction Error Across Ranks

As previously mentioned, the primary strength of NMF is interpretability, and this dataset provides a clear illustration. Each row H matrix corresponds to a latent component, and the heatmap shown below shows how strongly each original feature vector loads onto each component. Given that a good portion of our input consisted of one-hot encodings, many components reflect different subgroups of the dataset. For example, earlier inspection of component loadings revealed components dominated by specific manufacturers (such as Toyota or Chevrolet), regions, or vehicle types. These concentrated weight patterns demonstrate the parts-based nature of the decomposition. Where each component emphasizes a cluster of attributes rather than mixing and blending unrelated features.

In comparison to PCA, whose components are linear combinations of both positive and negative coefficients, NMF provides strictly nonnegative, additive feature clusters.

7.3. K-Deep Simplex. To evaluate our K-Deep Simplex model for used car price prediction, we performed the following 3 step pipeline. The purpose of this pipeline ensures that the Dictionary Learning Algorithm in Section 4, equation (4.2) is on the same scale as we obtain sufficient data.

- Step 1: Establish the Hidden Size using KMeans Clustering
- Step 2: Sample Random Probability Vectors to test different data reconstruction outputs.
- Step 3: Compute the Reconstruction to Locality Loss ratio C to check which term in the Dictionary Learning Algorithm is dominating.

7.3.1. *Establishing the Hidden Size using KMeans Clustering.* Once the training and testing data is setup, KMeans was applied to subdivide the training dataset into clusters. These clusters will represent the archetypes in the model training, separating the cars based on make, model, title status, etc.

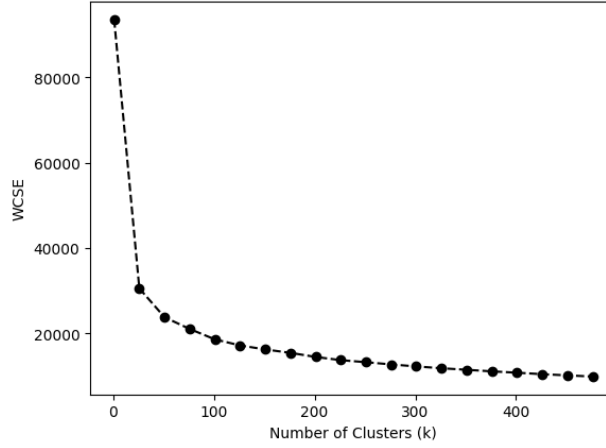


Figure 7.3.1.1: Number of Clusters vs. Weighted Sum of Square Errors

In this training, we consider $k = 90$.

7.3.2. *Sample Random Probability Vectors to test different data reconstruction outputs.* We leverage Kraemer's algorithm to randomly sample probability vectors in the probability simplex. As such, we follow its 3 step pipeline.

- Step 1: Select $x_1, \dots, x_{n-1}$ from a uniform distribution between 0 to 1, with $x_n = 1 - \sum_{i=1}^n x_i$.
- Step 2: Sort the x_i s in place
- Step 3: Compute the differences between the consecutive values.

This will be used to generate the sparse codes to perform the convex combination with the learned archetypes (cluster centers) to reconstruct our data points.

7.3.3. *Compute the Reconstruction to Locality Loss Ratio.* Once the reconstruction and locality loss is computed, we check to see which term is dominating. If the reconstruction loss is dominating, this implies that we fail to preserve the local reconstruction of the data points. Otherwise, if the locality loss is dominating, this implies that the model fails to learn the other relevant archetypes contributing to the data points. To ensure that both loss terms are on the same scale, $\lambda = C * t$, where $0 \leq t \leq 1$ is the multiplier that attempts to balance the reconstruction and locality loss.

We repeat these steps together across 15 trials, where we randomly subsample 10,000 training data points across the pipeline in order to determine the average reconstruction loss

to locality loss ratio, which we denote as C . We can use this adjust the parameter t to obtain an optimal representation of archetypes to construct our data matrix.

```
=====
FINAL C ESTIMATION RESULTS
Ratios across trials: [0.26142441 0.25952852 0.25924995 0.26442242 0.26244341 0.26385122
0.26140591 0.25984787 0.26076868 0.26006249 0.2638427 0.26325575
0.25843371 0.26354448 0.25998886]
Mean C: 0.261471 ± 0.001892
=====
```

Figure 7.3.3.1: Reconstruction to Locality Loss ratio across 15 experiments with the average mean and standard deviation.

7.3.4. Reconstruction to Locality Loss Ratio Results. From the resulting figure, we can see that the average ratio C is around 0.261471 with a standard deviation of around 0.001892. This tells us that the locality loss is around 3.8 times bigger than the reconstruction loss, indicating that the data points are reconstructed mainly by their local archetypes, not utilizing the archetypes outside the ones local to each data point.

7.3.5. K -Deep Simplex Results. For the model training, we used three different values for t : $1e^{-4}$, $1e^{-5}$, $5e^{-6}$ to achieve different values of lambda to emphasize locality. We trained each model with 100 epochs, with the number of encoding layers of the KDS network set to 15, with a learning rate set to $1e^{-3}$, and batch size set to 32.

To explore the sparsity patterns and active component usage with reconstructing our data points, we present three figures to illustrate the effects.

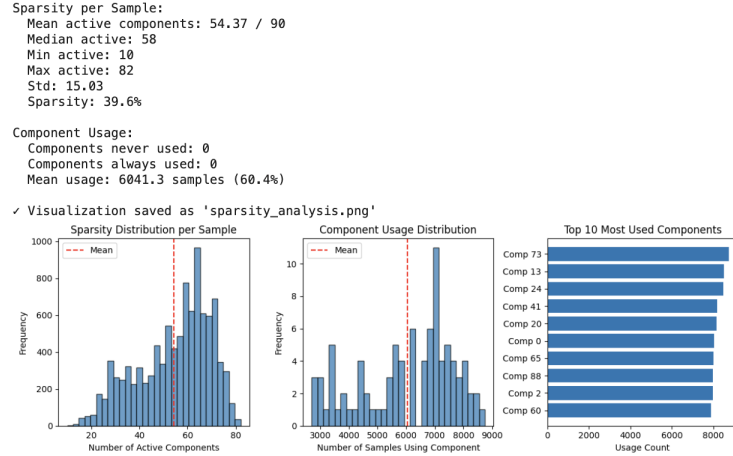


Figure 7.3.5.1: Sparse Distributions with the Model Parameter $t = 1e^{-4}$

This figure provides insight into how all the 90 components are used to approximate our training data. This includes the average number of components used to approximate each data point, the number of data points that use each component, and the 10 most frequent used components. From the first figure, we can use on average 54 components and that will successfully approximate each data point and each component is used on average on 6,041 samples. Across the top 10 most frequent components, each component is used equally with no component dominating the data reconstruction. We can infer that at $t = 1e^{-4}$, the learned KDS model achieves a great balance between reconstruction of the data point using its local archetypes.

COMPARATIVE SUMMARY				
Cluster	Label	Size	Price	Key Features
0	Economy	1,651 (16.5%)	\$14,500	Mixed
1	Economy	1,527 (15.3%)	\$13,995	Mixed
2	Economy	2,151 (21.5%)	\$13,000	Mixed
3	Economy	3,868 (38.7%)	\$14,000	Mixed
4	Economy	803 (8.0%)	\$14,888	Mixed

CLUSTER DISTINCTIVENESS ANALYSIS		
Features that differ most between clusters:		
Feature	F-Statistic	Significance
manufacturer_ram	2.41	*

*** p<0.001, ** p<0.01, * p<0.05

Figure 7.3.5.2: Subdivision of our training data using KMeans into 5 clusters with the Model Parameter $t = 1e^{-4}$ and sparsity tolerance level $1e^{-4}$

This summary provides a comprehensive insight into how the training data’s learned sparse codes at $t = 1e^{-4}$ is subdivided into 5 distinct clusters. The most important clusters are cluster 3 and cluster 4, highlighting the most and least representation across all 5 clusters. With our most frequent cluster, our average price is around \$14,000. It’s wide representation is clearly evident that we would expect a mix of low value and middle valued cars. With our least frequent cluster, our average price is around \$14,888. It’s low representation but higher average price is evident that we have middle valued to higher valued cars. Across each cluster, we can see that its size and price distributions is significantly different based on whether the car is manufactured by the make RAM.

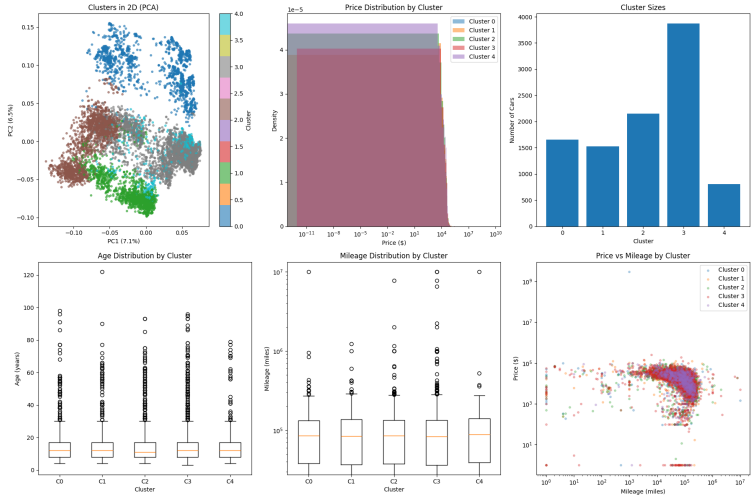


Figure 7.3.5.3: Clustering Analysis Results with the Model Parameter $t = 1e^{-4}$ and sparsity tolerance level $1e^{-5}$

These six figures highlight the current age, mileage, and price distributions, and the relationship between mileage and the price of a used car. From the previous figure, we concluded that cluster 4 despite its low representation has the highest average price. This is clearly evident in the price vs. mileage by cluster figure, where the mileage of the majority of its datapoints is between 10,000 to 100,000 miles. This indicates that cluster 4 has new and low mileage cars in the market, correlating perfectly with the clustering summary explained on the previous figure.

In addition, we concluded that cluster 3 despite its highest representation has one of the lowest average prices. This is clearly evident in the price vs. mileage by cluster figure, where the mileage of the majority of its datapoints is between 1,000 and exceeds 100,000 miles. This indicates that in cluster 3, we would expect to find a healthy mix of both high-mileage and low-valued cars and low-mileage and high-valued cars.

Thus, from these results we can conclude that there’s a negative correlation between the mileage and price. As the mileage increases, the price decreases. From a market standpoint, we can infer that the cars captured in cluster 3 will capture a wider market of customers due to its wider range of car selection from a dealership standpoint. Future work can explore the kinds of cars that are in each cluster to better capture the cause and effect of the car selection towards the size of the market.

8. COMPARISON OF METHODS

K-Deep Simplex Results		
Regression Model	RMSE	R^2
Linear Regression	• $t = 1e^4$: \$20388.93 • $t = 1e^5$: \$28295.52 • $t = 5e^6$: \$25898.76	• $t = 1e^4$: 0.4090 • $t = 1e^5$: 0.4090 • $t = 5e^6$: 0.4098
Histogram Gradient-Boosting Regressor	• $t = 1e^4$: \$17,732.07 • $t = 1e^5$: \$17,905.32 • $t = 5e^6$: \$17,682.53	• $t = 1e^4$: 0.5365 • $t = 1e^5$: 0.5386 • $t = 5e^6$: 0.5382

PCA Results	
RMSE	R^2
0.239	0.3410

NMF Results	
RMSE	R^2
Result	Result

9. CONCLUSION

REFERENCES

- [1] Abiy Tasissa. An algorithm implemented in python for local dictionary learning, 2025. Available at <https://github.com/abiy-tasissa/K-Deep-Simplex-KDS-algorithm>.
- [2] Austin Reese. Used cars dataset. kaggle.com, 2021. Available at <https://www.kaggle.com/datasets/austinreese/craigslist-carstrucks-data/data?select=vehicles.csv>.
- [3] Avi Chawla. The advantages and disadvantages of pca to consider before using it. Available at <https://blog.dailydoseofds.com/p/the-advantages-and-disadvantages/>.
- [4] Daniel D. Lee and H. Sebastian Seung. Algorithms for non-negative matrix factorization, 2000.
- [5] Noah Smith, Roy Tromble. Sampling uniformly from the unit simplex, August 2004. Available at <https://www.cs.cmu.edu/~nasmith/papers/smith+tromble.tr04.pdf>.
- [6] Abiy Tasissa, Pranay Tankala, James M. Murphy, and Demba Ba. K-deep simplex: Manifold learning via local dictionaries. 2024.

- [7] Your Data Teacher. Why you shouldn't use pca in a supervised machine learning project, July 2022. Available at <https://www.yourdatateacher.com/2022/07/11/why-you-shouldnt-use-pca-in-a-supervised-machine-learning-project>.